유비샘 4차 세미나

- https://vuetifyjs.com/en/getting-started/installation/
- https://vuetifyjs.com/en/components/all/
- https://velog.io/@rkddntjd7/Convention-over-ConfigurationCoC%EB%9E%80

> 💡 이번 세미나는 해당 URL 을 참조하면 좋습니다.

CRUD

- 자바코드로 search 쿼리 구현하기

- 테스트 코드로 이동합니다.

> 💡 저희는 ORM을 사용하기에 SQL 쿼리는 사용하지 않습니다.

- 그 후 아래 코드를 작성합니다.

  검색할 Hello를 of안에 Hello.class로 넣습니다.

```
@Test
void contextLoad3() throws Exception {
    JpaSpecificationBuilder<Hello> query = JpaSpecification
Builder.of(Hello.class);
}
```

```
//Search 쿼리 구현하기
@Test
void contextLoad3() throws Exception {
    JpaSpecificationBuilder<Hello> query = JpaSpecificationBuilder.of(type: Hello.class);
}
```

- 그 후 쿼리를 코드로 작성해 추가합니다.

- 아래 코드를 추가합니다.

  해당 코드는

  select * from hello where name = '홍길동' or email like '%abc@abc.com%'

  와 동일한 결과를 도출합니다.

```
query.where()
            .and()
            .eq("name", "홍길동")
            .or()
            .like("email", "%abc@abc.com%");
```

```
//Search 쿼리 구현하기
@Test
void contextLoad3() throws Exception {
    JpaSpecificationBuilder<Hello> query = JpaSpecificationBuilder.of(type: Hello

    //select * from hello where name = '홍길동' or email like '%abc@abc.com%'
    query.where()
        .and()
        .eq(property: "name", value: "홍길동")
        .or()
        .like(property: "email", value: "%abc@abc.com%");

}
```

- 추가로

  select * from hello where (name = '홍길동' and email like 'abc@abc.com') or (email like 'abc2@abc.com) 에 해당하는 쿼리를 코드로 짜면 아래와 같습니다.

```
query.where()
    .andStart().and().eq("name", "홍길동")
    .and().like("email", "abc@abc.com").andEnd()
    .or().like("email", "abc2@abc.com")
```

- 해당 쿼리 코드를 테스트 하기 위해 HelloRepository.java 로 이동합니다

  상속을 현재 JpaRepository로 받고있었으나 해당 상속을 RestfulJpaRepository로 변경합니다.

```java
package com.ubisam.example1.helloes;

import org.springframework.data.jpa.repository.JpaRepository;

import io.u2ware.common.data.jpa.repository.RestfulJpaRepository;


// public interface HelloRepository extends JpaRepository<Hello, Long>{
public interface HelloRepository extends RestfulJpaRepository<Hello, Long> {

    // List<Hello> findByEmail(String email);
    // List<Hello> findByNameAndEmail(String name, String email);
    // List<Hello> findByIdOrName(Long id, String name);
}
```

- 그 후 다시 테스트 코드로 이동하여 query.build()를 통해 빌드를 해줍니다.

```java
helloRepository.findAll(query.build());
```

아래 사진과 같이 저희는 findAll(Specification<Hello> spec)를 사용합니다.

```
findAll()                                                                                    Iterable<Hello>
findAll(Pageable pageable)                                                                   Page<Hello>
findAll(Sort sort)                                                                           Iterable<Hello>
findAll(Specification<Hello> spec)                                                           List<Hello>
findAll(Specification<Hello> spec, Pageable pageable)                                        Page<Hello>
findAll(Specification<Hello> spec, Sort sort)                                                List<Hello>
findAll(Specification<Hello> spec, Specification<Hello> countSpec, Pageable pageable)        Page<Hello>
findAllById(Iterable<Long> ids)                                                              Iterable<Hello>
findAll()                                                                                    Iterable<Hello>
findAll(Pageable pageable)                                                                   Page<Hello>
findAll(Sort sort)                                                                           Iterable<Hello>
findAll(Specification<Hello> spec)                                                           List<Hello>
```

- 테스트 코드 실행 시 아래와 같이 select 문이 정상적으로 작동하는 것을 확인가능합니다.

```
OpenJDK 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended
Hibernate:
    select
        h1_0.id,
        h1_0.email,
        h1_0.name
    from
        t_str h1_0
    where
        h1_0.email like '%abc@abc.com%' escape '\'
        or h1_0.name='홍길동'
```

- Handler에 beforeSearch를 만듭니다.

  아래와 같이 코드를 작성합니다.

```
@HandleBeforeRead
public void beforeSearch(Hello hello, Specification<Hello>
spec){
    System.out.println("[beforeSearch] testtesttesttesttest
testtesttest");

}
```

- 해당 코드의 작동을 확인하기 위해 사전작업을 준비합니다.
- pom.xml에 아래 의존성을 추가한 후 pom.xml을 우클릭하여 프로젝트를 리로드합니다.

  리로드 하였는데도 의존성 추가가 이루어지지 않았다면 VSCode를 재부팅합니다.

  ```
  <dependency>
      <groupId>com.querydsl</groupId>
      <artifactId>querydsl-jpa</artifactId>
  </dependency>
  ```

  그후 Example1Application.java에

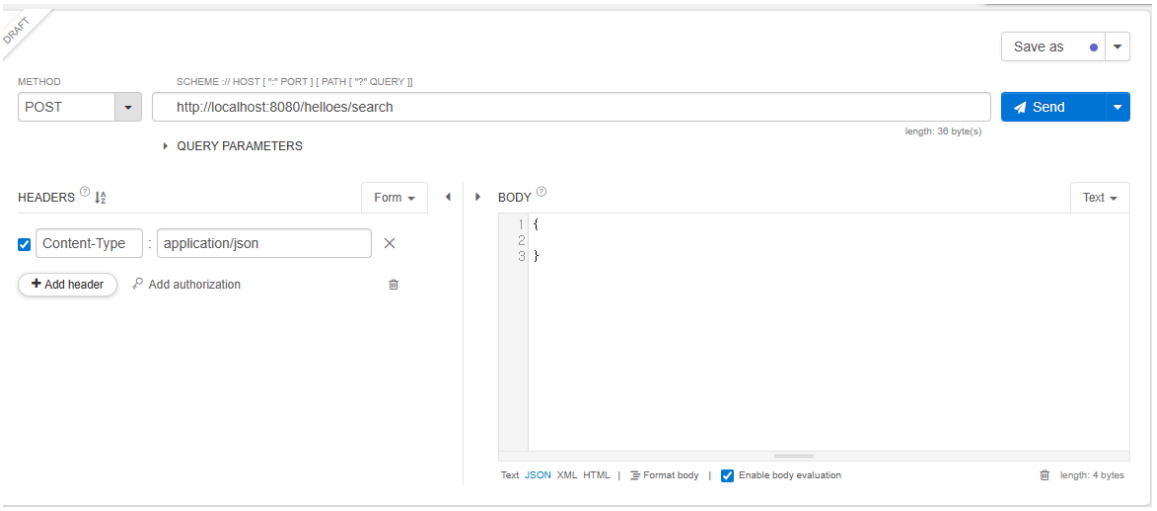
  ```
  import org.springframework.data.jpa.repository.config.En
  ableJpaRepositories;
  import io.u2ware.common.data.jpa.config.EnableRestfulJpa
  Repositories;

  @EnableRestfulJpaRepositories
  @EnableJpaRepositories
  ```

  해당 설정을 추가합니다.

```java
package com.ubisam.example1;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.data.jpa.repository.config.EnableJpaRepositories;
import io.u2ware.common.data.jpa.config.EnableRestfulJpaRepositories;

@EnableRestfulJpaRepositories
@EnableJpaRepositories
@SpringBootApplication
public class Example1Application {

    public static void main(String[] args) {
        SpringApplication.run(primarySource: Example1Application.class, args);
    }

}
```

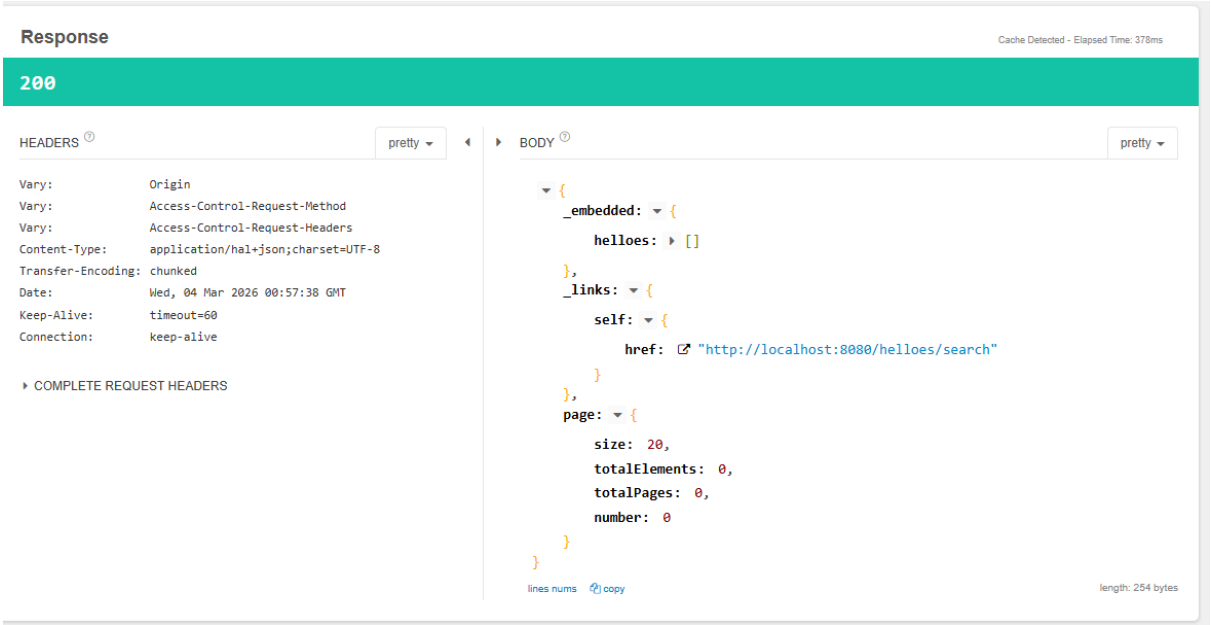
- ./mvnw spring-boot:run을 통하여 서버를 동작합니다.
- Talend API를 활용하여 테스트 작업을 수행합니다.

  아래 사진과 같이 POST요청을 보냅니다.

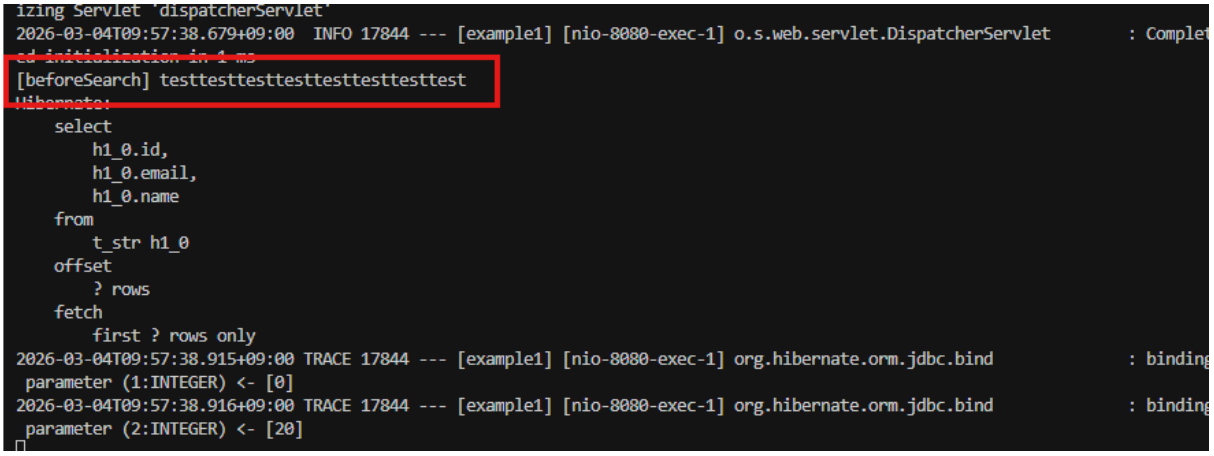


- 그럼 아래 사진과 같이 결과가 찍히게 됩니다.

  현재 DB에 아무것도 Create한게 없기때문에 helloes는 []로 출력됩니다.



- 콘솔에서도 아래 사진과 같이 Handler에 추가한 코드 확인이 가능합니다.



사용자의 입력을 파라미터에 입력하기

- 저희가 검색할 것은 실제 DB 속성이 아니라 검색어의 속성입니다.

  그렇기에 keyword 속성을 추가해야 합니다.

  Hello.java로 이동합니다.

  아래와 같이 keyword 속성을 추가합니다.

```
import jakarta.persistence.Transient;

@Transient
private String keyword;
```

💡 @Transient : DB에 저장되지 않는 필드

- 이제 Handler로 가서 확인을 합니다.

  아래 코드를 추가합니다.

```
JpaSpecificationBuilder<Hello> query = JpaSpecificationBuil
der.of(Hello.class);
  query.where()
      .and().eq("name", hello.getKeyword()).build(spec);
```

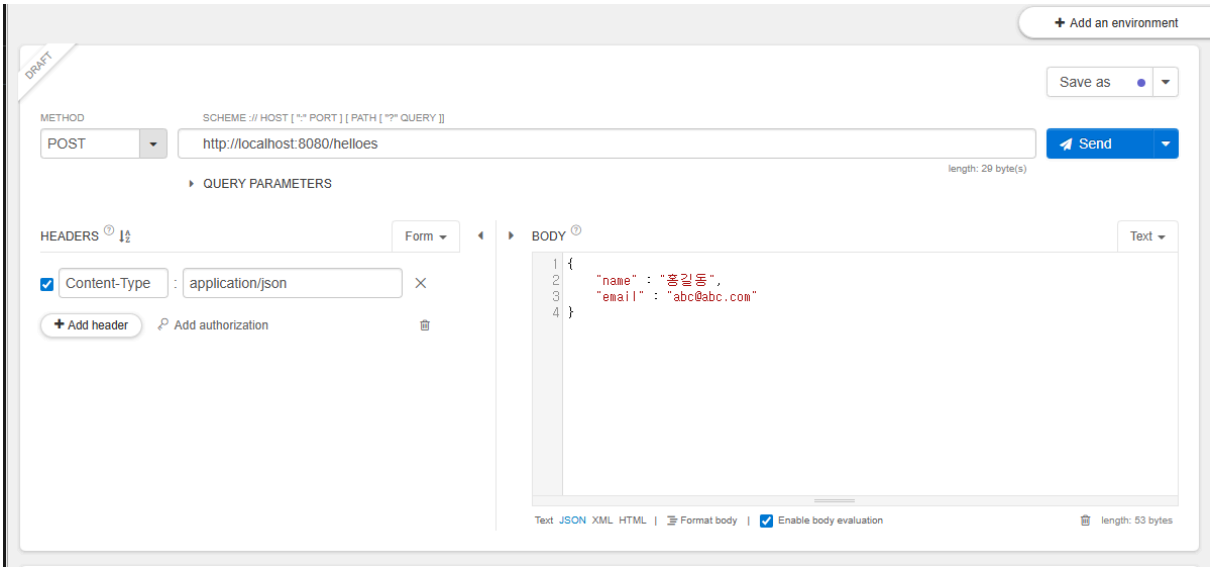
- 최종적으로 beforeSearch 메소드는 아래와 같이 됩니다.

```
@HandleBeforeRead
public void beforeSearch(Hello hello, Specification<Hello>
spec){
    System.out.println("[beforeSearch] testtesttesttesttest
testtesttest");

    JpaSpecificationBuilder<Hello> query = JpaSpecification
Builder.of(Hello.class);
    query.where()
        .and().eq("name", hello.getKeyword()).build(spec);

}
```

```
@HandleBeforeRead
public void beforeSearch(Hello hello, Specification<Hello> spec){
    System.out.println(x: "[beforeSearch] testtesttesttesttesttesttesttest");

    JpaSpecificationBuilder<Hello> query = JpaSpecificationBuilder.of(type: Hello.class);
    query.where()
        .and().eq(property: "name", hello.getKeyword()).build(spec);

}
```

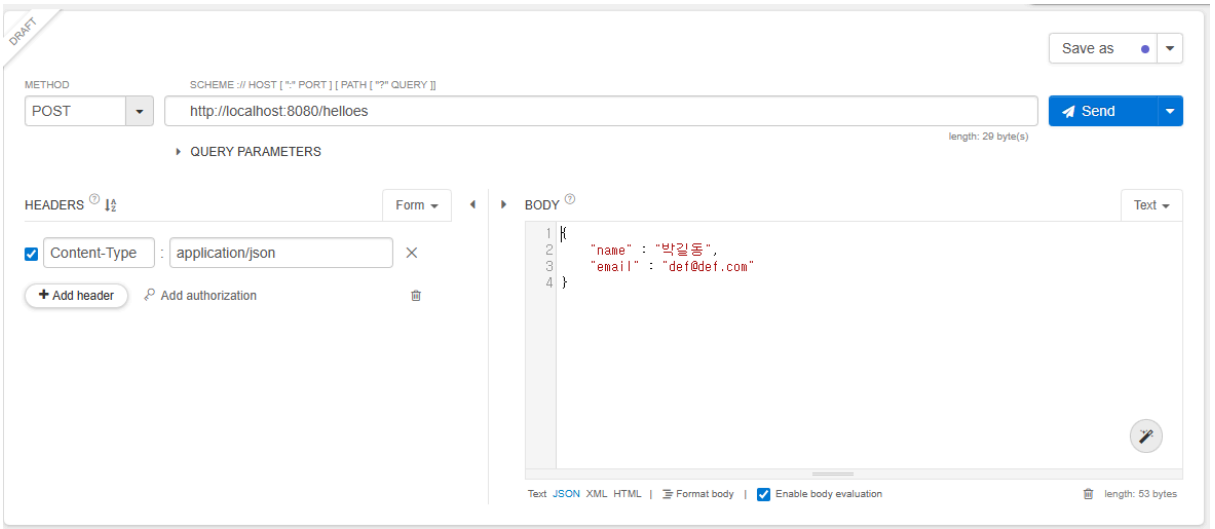
- 이제 다시 ./mvnw spring-boot:run을 통하여 서버를 동작합니다.

  Search를 하기 위해 먼저 Talend을 열어서 아래 사진과 같이 입력 후 POST를 보냅니다.



- 그럼 201 Response가 나옴과 동시에 Create문을 실행하였습니다.

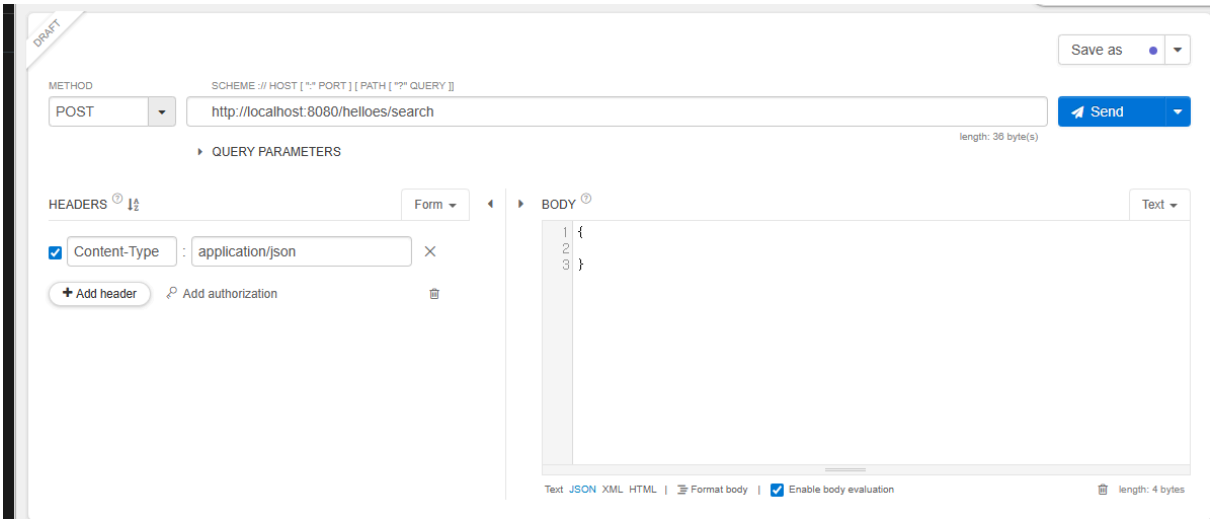
- 다시 아래와 같이 1개 더 입력해줍니다.

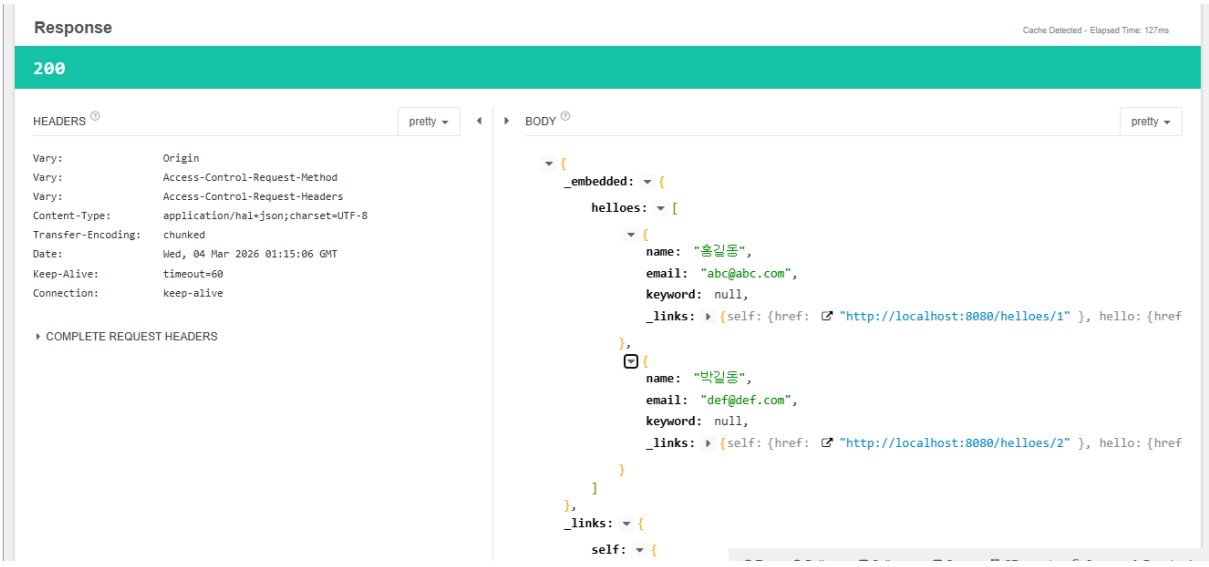


- 그럼 똑같이 입력된 것을 확인하실 수 있습니다.

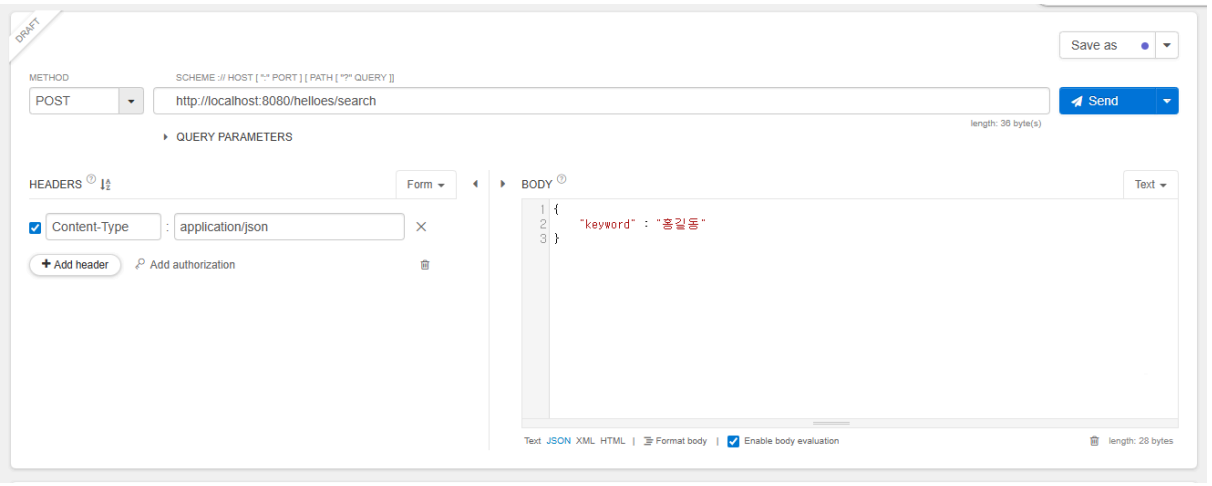
- 이제 Search로 확인을 해보면 아래 사진과 같이

  총 2개의 데이터가 들어가 있는 것을 확인해볼 수 있습니다.

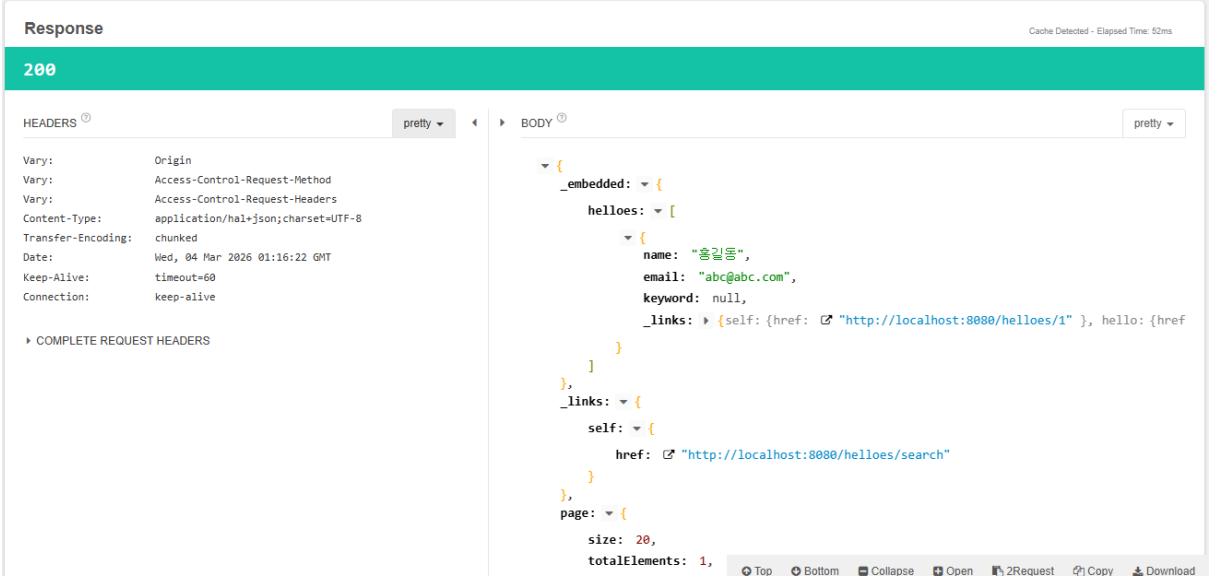




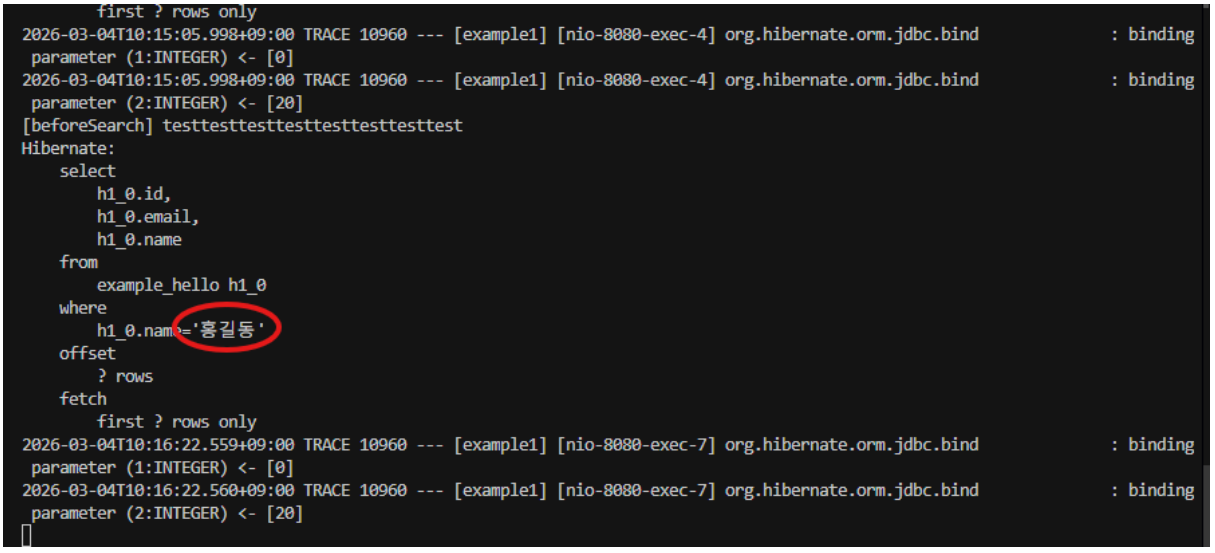
- 그럼 이제 keyword를 확인하기 위하여 아래와 같이 Talend에 입력합니다.



- Send를 누르게 되면 아래와 같이 200 응답과 함께 홍길동으로 검색된 것을 확인 가능합니다.



- 또한 콘솔에서 파라미터 값에 '홍길동' 이 들어간 것도 확인이 가능합니다.



- 더 자세한 테스트를 위해 System.out.println을 넣어보겠습니다.

```java
@HandleBeforeRead
public void beforeSearch(Hello hello, Specification<Hello> spec){
    System.out.println(x: "[beforeSearch] testtesttesttesttesttesttesttest");

    JpaSpecificationBuilder<Hello> query = JpaSpecificationBuilder.of(type: Hello.class);
    query.where()
        .and().eq(property: "name", hello.getKeyword()).build(spec);

    System.out.println("[getKeyword] " + hello.getKeyword());
}
```

- 콘솔에서 아래와 같이 getKeyword가 출력되는 것을 확인하실 수 있습니다.

```
[beforeSearch] testtesttesttesttesttesttesttest
[getKeyword] 박길동
Hibernate:
    select
        h1_0.id,
        h1_0.email,
        h1_0.name
    from
        example_hello h1_0
    where
        h1_0.name='박길동'
    offset
        ? rows
    fetch
        first ? rows only
2026-03-04T10:26:46.777+09:00 TRACE 7572 --- [example1] [nio-8080-exec-4] org.hibernate.orm.jdbc.bind            : binding
parameter (1:INTEGER) <- [0]
2026-03-04T10:26:46.792+09:00 TRACE 7572 --- [example1] [nio-8080-exec-4] org.hibernate.orm.jdbc.bind            : binding
parameter (2:INTEGER) <- [20]
```

- 이제 웹으로도 테스트를 해보겠습니다.

  Example1ApplicationTests.java에 contextLoad3 맨 아래 부분에

  아래와 같이 코드를 추가합니다.

```java
Hello h = new Hello();
h.setKeyword("홍길동");

mockMvc.perform(post("/helloes/search")
        .content(h))
        .andDo(print())
        .andExpect(is2xx());
```

```java
@Test
void contextLoad3() throws Exception {
    JpaSpecificationBuilder<Hello> query = JpaSpecificationBuilder.of(type: Hello.class);

    //select * from hello where name = '홍길동' or email like '%abc@abc.com%'
    query.where()
        .and()
        .eq(property: "name", value: "홍길동")
        .or()
        .like(property: "email", value: "%abc@abc.com%");
    /*
    ==========================================
     select * from hello where
     (name = '홍길동' and email like 'abc@abc.com')
     or (email like 'abc2@abc.com')

     query.where()
        .andStart().and().eq("name", "홍길동")
        .and().like("email", "abc@abc.com").andEnd()
        .or().like("email", "abc2@abc.com");
    ==========================================
     */
    helloRepository.findAll(query.build());

    Hello h = new Hello();
    h.setKeyword(keyword: "홍길동");

    mockMvc.perform(post(uri: "/helloes/search")
            .content(h))
            .andDo(print())
            .andExpect(is2xx());
}
```

- 그 후 테스트 코드를 실행합니다.

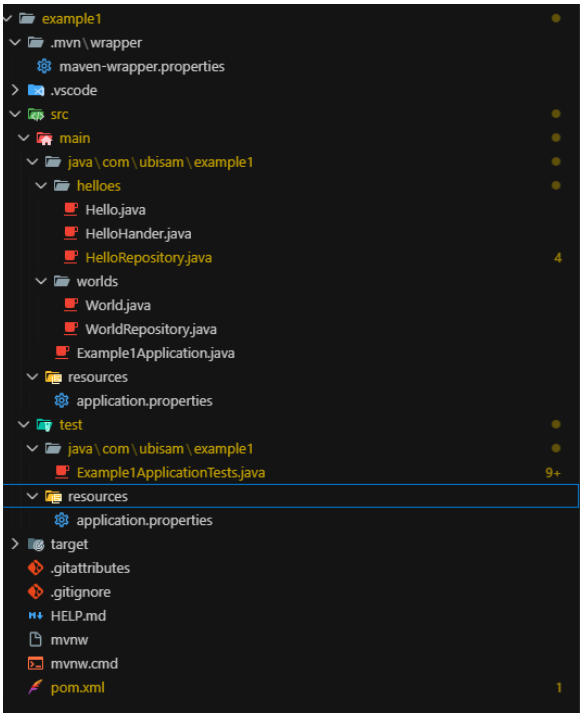
- 데이터가 없기 때문에 []가 출력되는 형태로 정상적으로 작동하는걸 확인하실 수 있습니다.

```
http://localhost/helloes/search
----------------------------------------

MockHttpServletRequest:
      HTTP Method = POST
      Request URI = /helloes/search
       Parameters = {}
          Headers = [Content-Type:"application/json;charset=UTF-8"]
             Body = {"id":null,"name":null,"email":null,"keyword":"홍길동"}
    Session Attrs = {}

Handler:
             Type = io.u2ware.common.data.jpa.webmvc.RestfulJpaRepositoryController
           Method = io.u2ware.common.data.jpa.webmvc.RestfulJpaRepositoryController#getCollectionResource(RootResourceInformation, String, boolean, DefaultedPageable, Sort, Map, PersistentEntityResourceAssembler)

Async:
    Async started = false
     Async result = null

Resolved Exception:
             Type = null

ModelAndView:
        View name = null
             View = null
            Model = null

FlashMap:
        Attributes = null

MockHttpServletResponse:
           Status = 200
    Error message = null
          Headers = [Vary:"Origin", "Access-Control-Request-Method", "Access-Control-Request-Headers", Content-Type:"application/hal+json;charset=UTF-8"]
     Content type = application/hal+json;charset=UTF-8
             Body = {
  "_embedded" : {
    "helloes" : [ ]
  },
  "_links" : {
    "self" : {
      "href" : "http://localhost/helloes/search"
    }
  },
  "page" : {
    "size" : 20,
    "totalElements" : 0,
    "totalPages" : 0,
    "number" : 0
  }
}
    Forwarded URL = null
   Redirected URL = null
          Cookies = []
----------------------------------------
```

COC(Convention over Configuration)

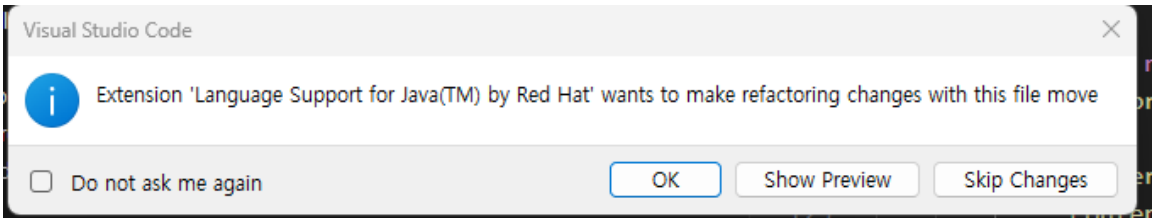
- 현재 아래와 같이 되어있는 구조를 사내

  COC(Convention over Configuration)에 맞게 변경하겠습니다.



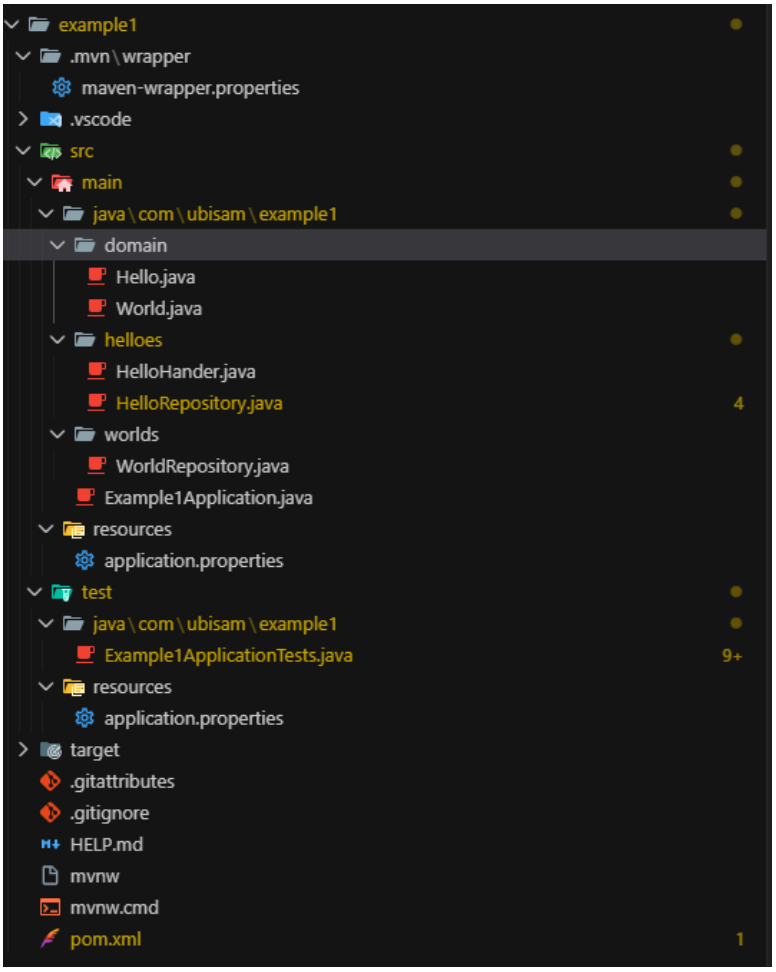
- 먼저 example1 폴더 밑에 domain이라는 폴더를 생성합니다.

  그 후 Hello.java와 World.java 파일을 domain 폴더 안으로 이동합니다.



  해당 메세지는 OK 누르시면 됩니다.

- 그럼 아래 와 같은 상태가 됩니다.



- 다음으로 example1 밑에 api라는 폴더를 생성합니다.

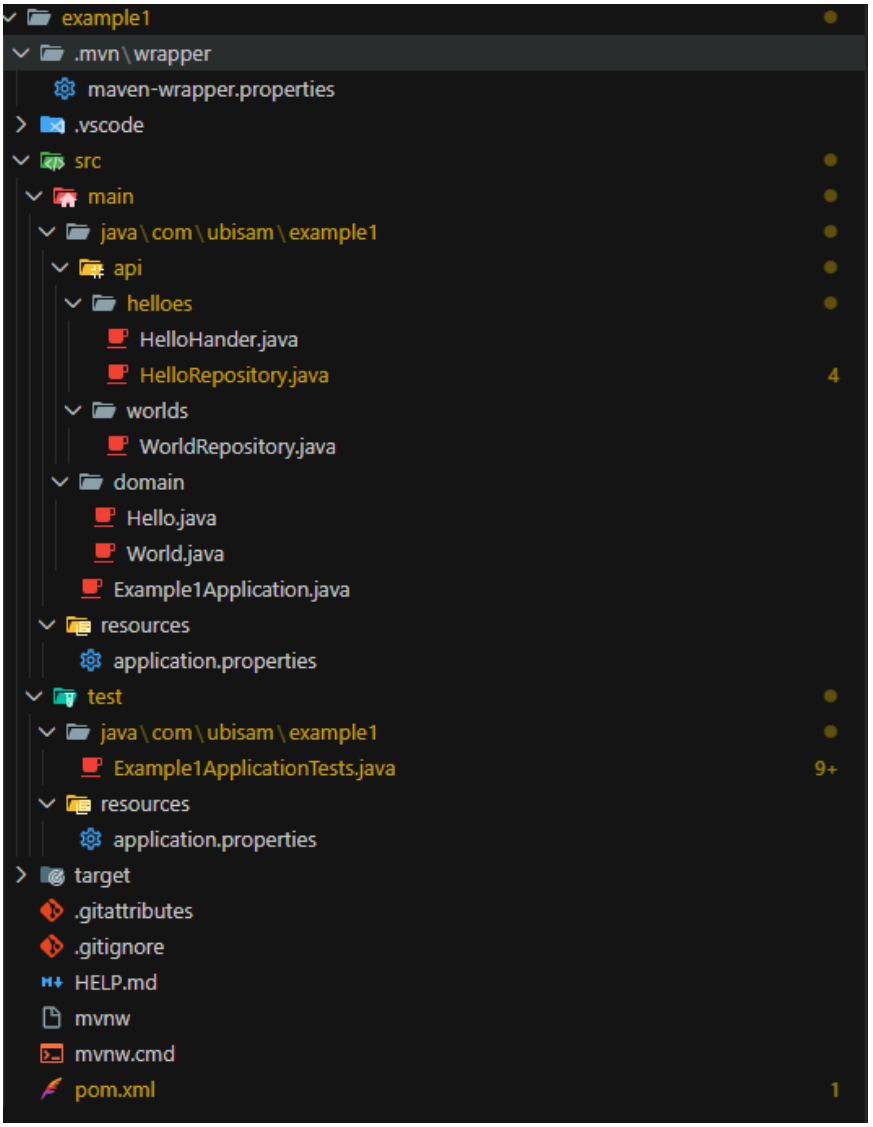
  해당 폴더 밑에 저희가 만들었던 hello와 world를 각각

  helloes, worlds 라는 폴더로 생성합니다.

  helloes 폴더 안에는 HelloHandler.java, HelloRepository.java를

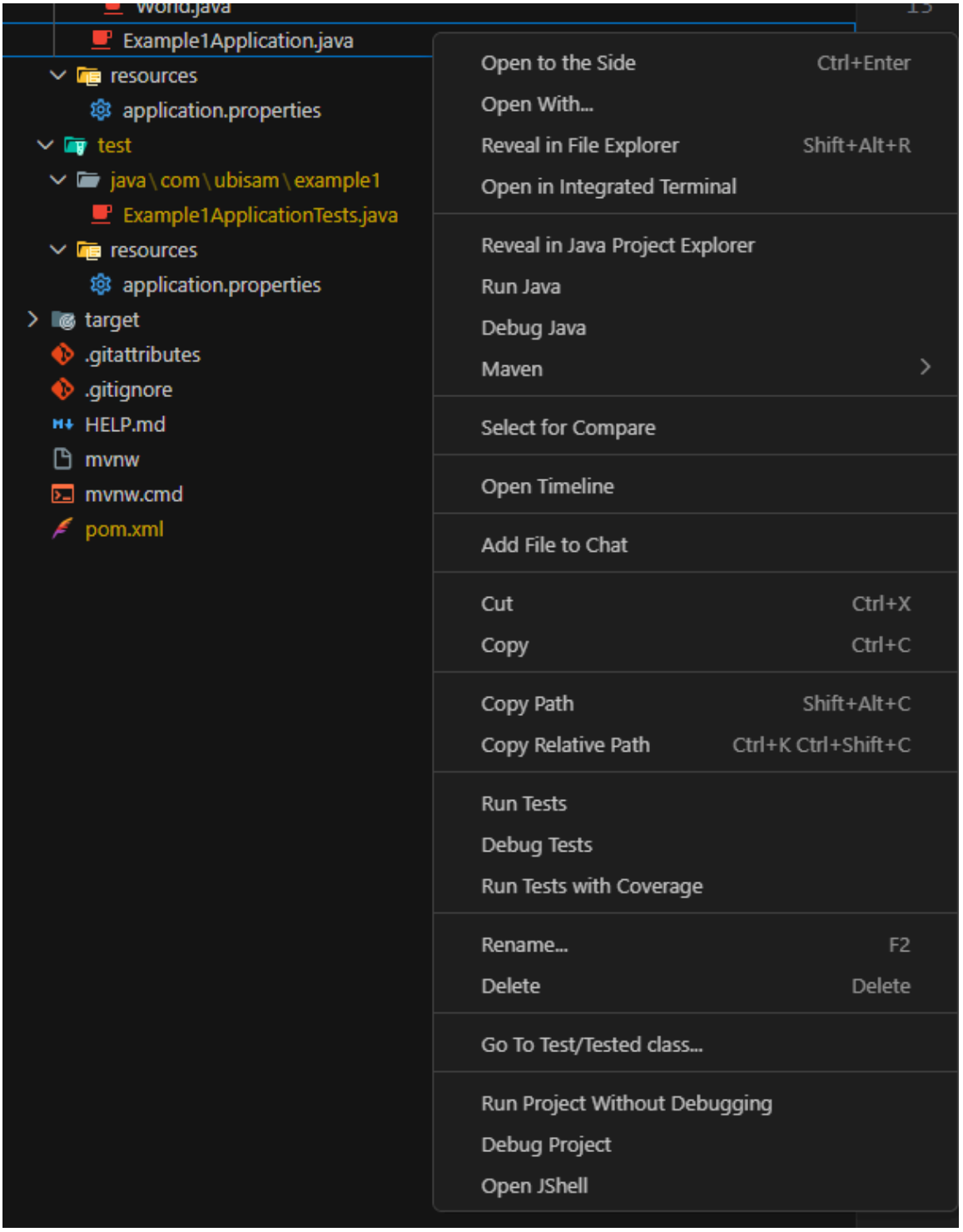
  worlds 폴더 안에는 WorldRepository.java를 이동합니다.

- 해당 작업이 끝나면 아래 사진과 같이 변경됩니다.



- 다음으로 Example1Application.java를 Application.java로 변경합니다.

  파일 → 우클릭 → Rename → Application.java로 변경



- Config 파일을 추가합니다. Application.java와 같은 위상에 ApplicationApiConfig.java를 생성합니다.

  그 후 아래와 같이 코드를 작성해줍니다.

```
import org.springframework.context.annotation.Configurat
ion;
import org.springframework.data.jpa.repository.config.En
```

```
ableJpaRepositories;
import org.springframework.data.rest.core.config.Reposit
oryRestConfiguration;
import org.springframework.data.rest.webmvc.config.Repos
itoryRestConfigurer;
import org.springframework.web.servlet.config.annotatio
n.CorsRegistry;

import com.ubisam.example2.domain.Hello;
import com.ubisam.example2.domain.World;

import io.u2ware.common.data.jpa.config.EnableRestfulJpa
Repositories;

@Configuration
@EnableRestfulJpaRepositories
@EnableJpaRepositories
public class ApplicationApiConfig implements RepositoryR
estConfigurer {

    @Override
    public void configureRepositoryRestConfiguration(Rep
ositoryRestConfiguration config, CorsRegistry cors) {
        // /api 주소 설정
        config.setBasePath("/api");
        // id 노출 설정
        config.exposeIdsFor(Hello.class, World.class);
        // CORS 설정 - 모든 주소 허용, 모든 메소드 허용, 쿠키
허용 안함, maxAge 설정
        cors.addMapping("/**")
            .allowedOrigins("*")
            .allowedMethods("*")
            .allowCredentials(false)
            .maxAge(999999);
    }
}
```

```java
package com.ubisam.example1;

import org.springframework.context.annotation.Configuration;
import org.springframework.data.jpa.repository.config.EnableJpaRepositories;
import org.springframework.data.rest.core.config.RepositoryRestConfiguration;
import org.springframework.data.rest.webmvc.config.RepositoryRestConfigurer;
import org.springframework.web.servlet.config.annotation.CorsRegistry;

import com.ubisam.example1.domain.Hello;
import com.ubisam.example1.domain.World;

import io.u2ware.common.data.jpa.config.EnableRestfulJpaRepositories;

@Configuration
@EnableRestfulJpaRepositories
@EnableJpaRepositories
public class ApplicationApiConfig implements RepositoryRestConfigurer {


    @Override
    public void configureRepositoryRestConfiguration(RepositoryRestConfiguration config, CorsRegistry cors) {
        // /api 주소 설정
        config.setBasePath(basePath: "/api");
        // id 노출 설정
        config.exposeIdsFor(...domainTypes: Hello.class, World.class);
        // CORS 설정 - 모든 주소 허용, 모든 메소드 허용, 쿠키 허용 안함, maxAge 설정
        cors.addMapping("/**")
            .allowedOrigins("*")
            .allowedMethods("*")
            .allowCredentials(false)
            .maxAge(999999);
    }
}
```

Application.java에 있는

@EnableRestfulJpaRepositories, @EnableJpaRepositories 어노테이션은 없애줍니다.

```java
package com.ubisam.example1;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class Application {

    public static void main(String[] args) {
        SpringApplication.run(primarySource: Application.class, args);
    }

}
```

- CORS란?

AI 개요

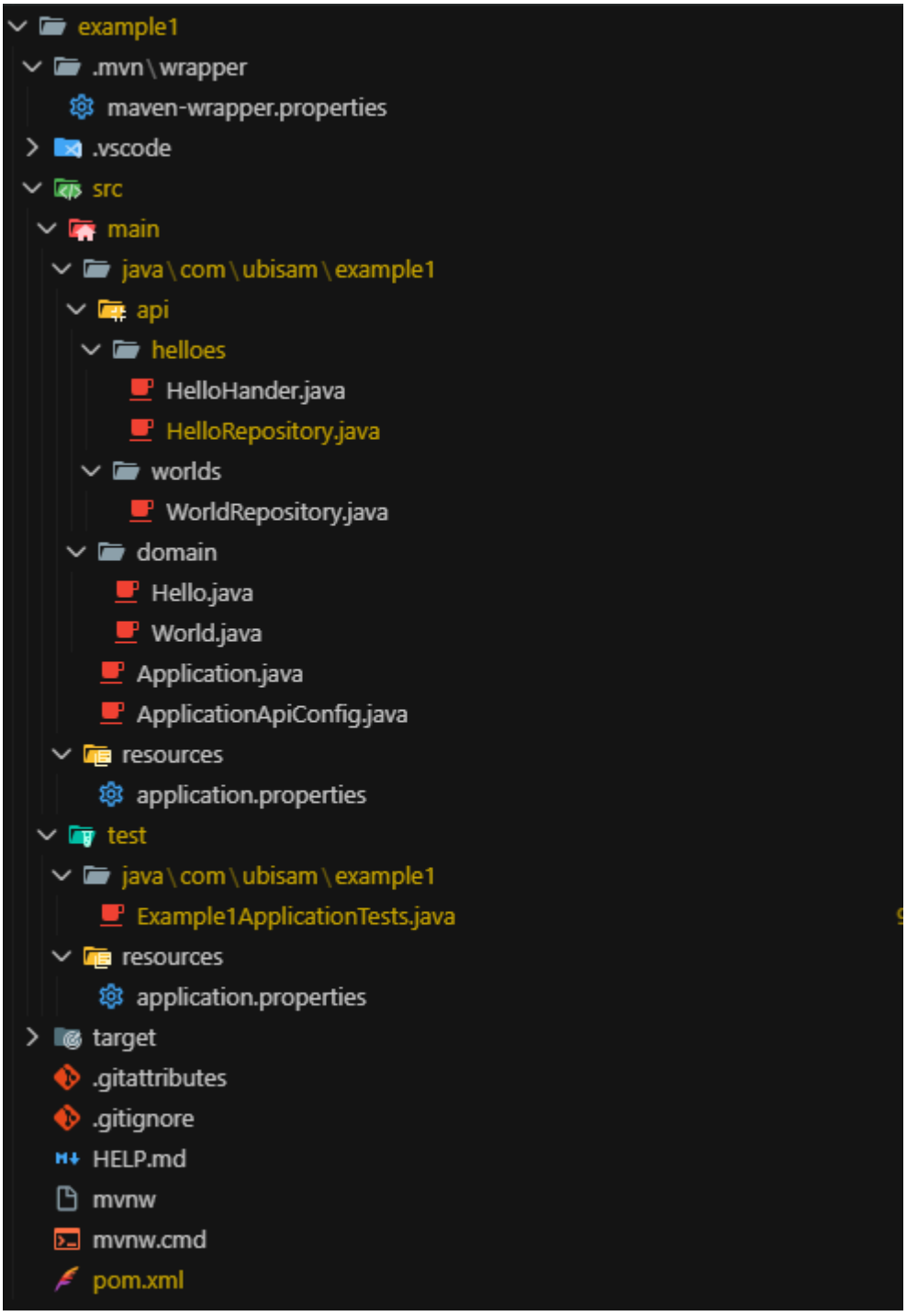
CORS(Cross-Origin Resource Sharing, 교차 출처 리소스 공유)는 **웹 브라우저가 보안상 다른 도메인(출처)의 서버로 API 요청을 보낼 때, 이를 허용하거나 차단하는 메커니즘**입니다. 동일 출처 정책(SOP)의 제약을 해결하며, 서버에서 `Access-Control-Allow-Origin` 헤더를 통해 특정 도메인의 접근을 허용하는 방식으로 작동합니다. YouTube +2

- 저희가 정한 기본경로는 /api 이므로 setBasePath에 /api를 넣어줍니다.
- Hello.class 와 World.class의 id를 노출되게 설정합니다.

  REST 응답구조에 ID는 원래 찍히지 않지만 해당 설정을 추가하면 찍히게 됩니다.

- 도메인을 묶은 이유는 관계가 있기 때문이며 API로 묶은 이유는 상단의 주소를 통일하기 위해서입니다.
  즉, 모든 백엔드의 구조는 도메인과 API로 지정합니다.
  폴더 구조는 많은 정보를 담고있으며 위와 같이 알아볼 수 있게 잘 설정해야 합니다.

- 최종적 구조는 아래 사진과 같습니다.



설정 완료 후 ./mvnw spring-boot:run 으로 서버를 켜서 정상작동 하는지 테스트합니다.

정상적으로 열리면 백엔드 작업은 끝이 납니다.

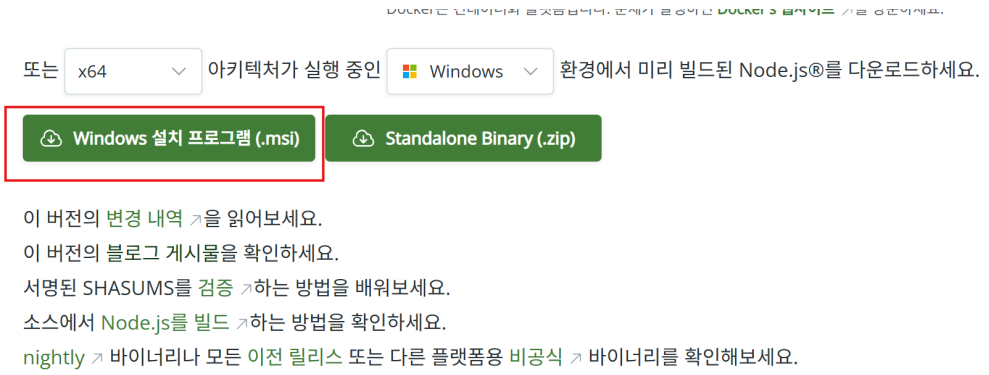
```
Initializing Spring embedded WebApplicationContext
2026-03-04T11:26:01.623+09:00  INFO 14656 --- [example1] [           main] w.s.c.ServletWebServerApplicationContext :
Root WebApplicationContext: initialization completed in 1499 ms
2026-03-04T11:26:01.881+09:00  INFO 14656 --- [example1] [           main] o.hibernate.jpa.internal.util.LogHelper  :
HHH000204: Processing PersistenceUnitInfo [name: default]
2026-03-04T11:26:01.941+09:00  INFO 14656 --- [example1] [           main] org.hibernate.Version                    :
HHH000412: Hibernate ORM core version 6.6.41.Final
2026-03-04T11:26:01.976+09:00  INFO 14656 --- [example1] [           main] o.h.c.internal.RegionFactoryInitiator    :
HHH000026: Second-level cache disabled
2026-03-04T11:26:02.190+09:00  INFO 14656 --- [example1] [           main] o.s.o.j.p.SpringPersistenceUnitInfo      :
No LoadTimeWeaver setup: ignoring JPA class transformer
2026-03-04T11:26:02.216+09:00  INFO 14656 --- [example1] [           main] com.zaxxer.hikari.HikariDataSource       :
HikariPool-1 - Starting...
2026-03-04T11:26:02.516+09:00  INFO 14656 --- [example1] [           main] com.zaxxer.hikari.pool.HikariPool        :
HikariPool-1 - Added connection org.postgresql.jdbc.PgConnection@289a4b90
2026-03-04T11:26:02.517+09:00  INFO 14656 --- [example1] [           main] com.zaxxer.hikari.HikariDataSource       :
HikariPool-1 - Start completed.
2026-03-04T11:26:02.549+09:00  WARN 14656 --- [example1] [           main] org.hibernate.orm.deprecation            :
HHH90000025: PostgreSQLDialect does not need to be specified explicitly using 'hibernate.dialect' (remove the propert
 setting and it will be selected by default)
2026-03-04T11:26:02.568+09:00  INFO 14656 --- [example1] [           main] org.hibernate.orm.connections.pooling    :
HHH10001005: Database info:
        Database JDBC URL [Connecting through datasource 'HikariDataSource (HikariPool-1)']
        Database driver: undefined/unknown
        Database version: 16.4
        Autocommit mode: undefined/unknown
        Isolation level: undefined/unknown
        Minimum pool size: undefined/unknown
        Maximum pool size: undefined/unknown
2026-03-04T11:26:03.173+09:00  INFO 14656 --- [example1] [           main] o.h.e.t.j.p.i.JtaPlatformInitiator       :
HHH000489: No JTA platform available (set 'hibernate.transaction.jta.platform' to enable JTA platform integration)
2026-03-04T11:26:03.223+09:00  INFO 14656 --- [example1] [           main] j.LocalContainerEntityManagerFactoryBean :
Initialized JPA EntityManagerFactory for persistence unit 'default'
2026-03-04T11:26:03.485+09:00  WARN 14656 --- [example1] [           main] JpaBaseConfiguration$JpaWebConfiguration :
spring.jpa.open-in-view is enabled by default. Therefore, database queries may be performed during view rendering. Ex
licitly configure spring.jpa.open-in-view to disable this warning
2026-03-04T11:26:03.957+09:00  INFO 14656 --- [example1] [           main] o.s.b.a.e.web.EndpointLinksResolver      :
Exposing 1 endpoint beneath base path '/actuator'
2026-03-04T11:26:04.022+09:00  INFO 14656 --- [example1] [           main] o.s.b.w.embedded.tomcat.TomcatWebServer  :
Tomcat started on port 8080 (http) with context path '/'
2026-03-04T11:26:04.030+09:00  INFO 14656 --- [example1] [           main] com.ubisam.example1.Application          :
Started Application in 4.235 seconds (process running for 4.443)
```

Vue

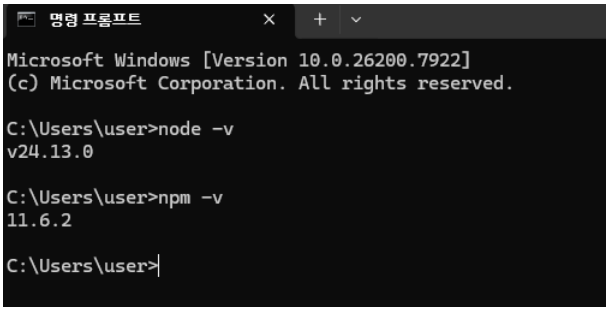
Node.js 설치하기

https://nodejs.org/ko/download

위 사이트에 가셔서 해당 프로그램을 설치합니다.

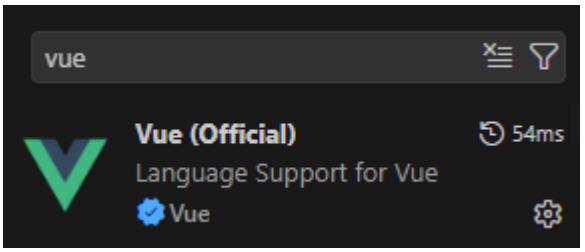


- 설치 완료 후 명령프롬포트에 node -v와 npm -v를 입력합니다. 오른쪽 사진과 같이 버전이 출력되면 정상입니다.



Vue Extensions 설치

- VSCode Extension에서 vue를 검색 후 Vue (Official)을 설치합니다.



Vuetify를 이용하여 프론트 생성하기

- https://vuetifyjs.com/en/getting-started/installation/#installation
  위 링크로 들어가서 아래 코드 npm create vuetify@latest를 복사합니다.

For more information regarding supported package managers, please visit their official websites:

- pnpm
- yarn
- npm
- bun

Using Vite

To get started with Vuetify 4, simply paste the following code into your terminal:

pnpm yarn npm bun

```
npm create vuetify@latest
```

- 해당 명령어를 백엔드 프로젝트 부모 폴더(사진 기준 test)위치에서 실행해줍니다.

```
PS C:\Users\user\Desktop\CHJ\notion용 사진\test>
PS C:\Users\user\Desktop\CHJ\notion용 사진\test> npm create vuetify@latest
```

- 아래와 같이 설정을 입력, 선택해줍니다. (긍정적으로 대답해주시면 됩니다 🙂)
  - frontend1
  - Vue
  - None
  - ESLint, Router, I18n
  - Vue Router (File-based)
  - Yes
  - npm
  - No (여기만 부정)

```
PS C:\Users\user\Desktop\CHJ\notion용 사진\test> npm create vuetify@latest
     T$$$$$$i$$$$`   .$$$l$$$$$$$:
      T$$$$ii$$$`   .$$$$ll$$$$$:
       `$$$l$$F`   :$$$$$$i$$$$`
         T$i$F    :$$$$$$$l$$F
          `:F    :$$$$$$$$i$:
           `    :$$$$$$$$l$`
               :$$$$$$$$j$`
               `$$$$$$$j$`
                 T$$$$l$`
                  `$$i$`
                   `T:

  Create Vuetify v3.0.3

◇ Project name:
  frontend1

◇ Which framework would you like to use?
  Vue

◇ Which CSS framework?
  None

◇ Select features to install: ↑/↓ to navigate, space to select, a to toggle all, enter to confirm
  ESLint, Router, I18n

◇ Which router would you like to use?
  Vue Router (File-based)

◇ Do you want to install dependencies?
  Yes

◇ Which package manager would you like to use?
  npm

◆ Do you want to save these settings as a preset?
  ○ Yes / ● No
```

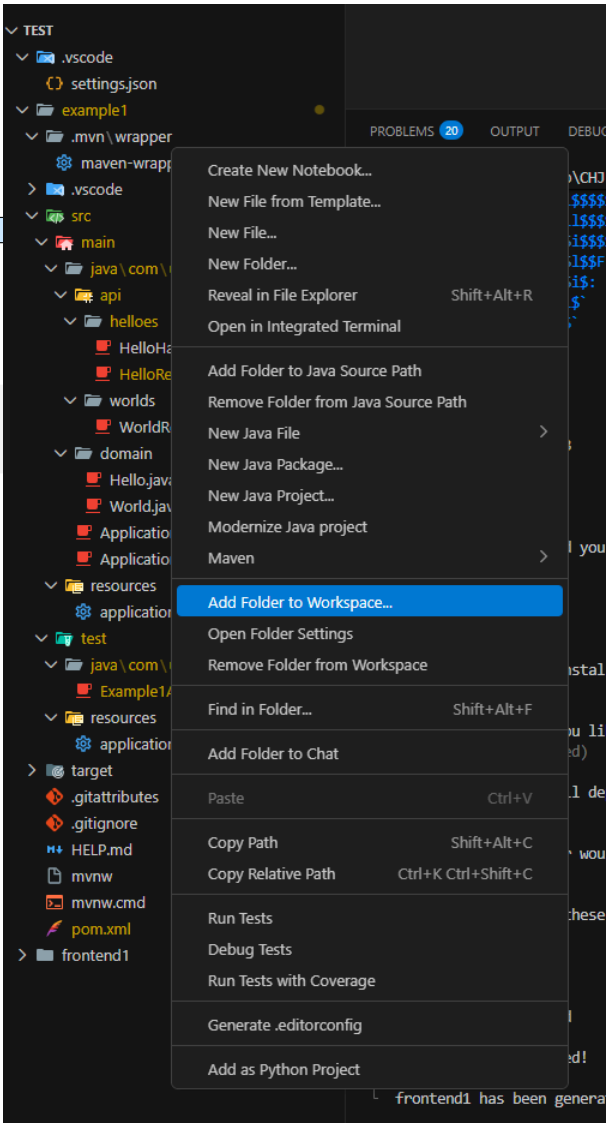
- 프로젝트 설치가 완료되면 아래 사진과 같이 나타납니다.

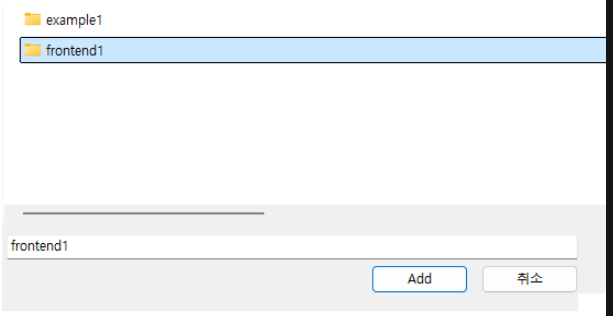
```
  No

◇ Template downloaded

◇ Configuration applied

◇ Dependencies installed!

  frontend1 has been generated at frontend1

 Next steps
  cd frontend1
  npm run dev

Docs · Discord · Support Us

PS C:\Users\user\Desktop\CHJ\notion용 사진\test>
```

- 해당 프로젝트를 VSCode에서 백엔드와 같이 사용해보겠습니다

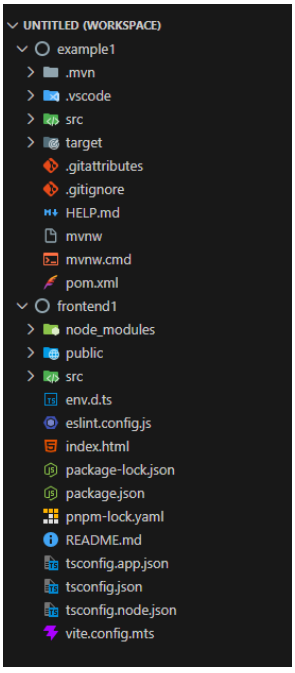
- 현재 프로젝트 빈칸 우클릭 후

  Add Folder to WorkSpace를 누릅니다.



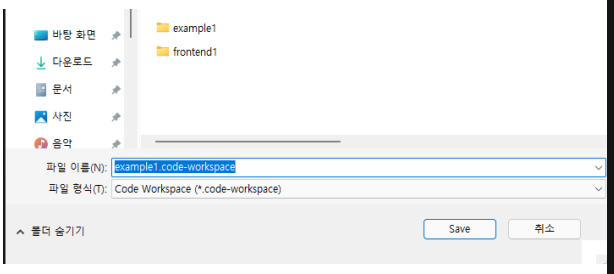


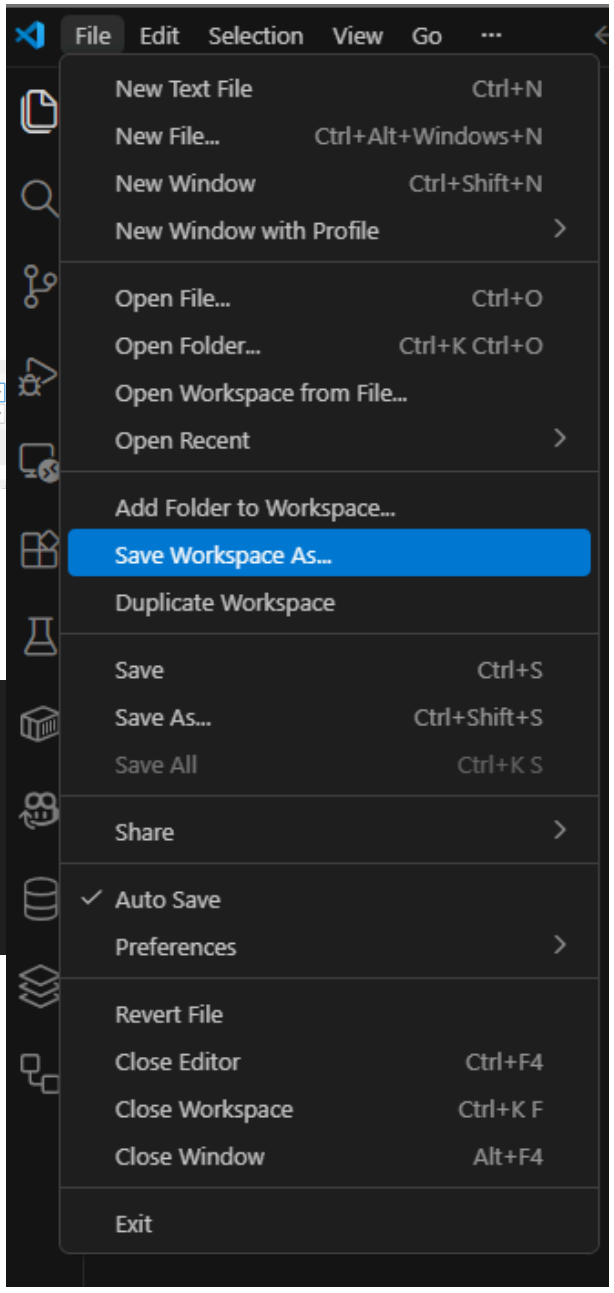
- 아래와 같이 frontend1을 선택 후 Add를 누릅니다.



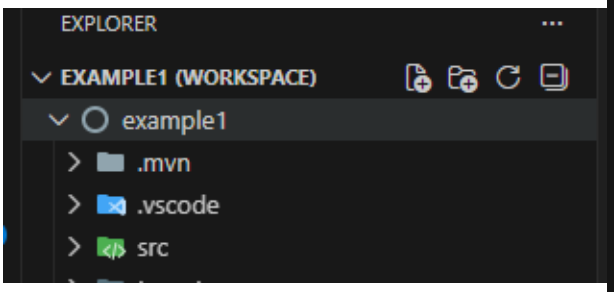
- 왼쪽 사진과 같이 UNTITLED(WORKSPACE)가 나오고 example1, frontend1이 추가되었으면 성공입니다.

- 그 후 File → Save Workspace As... 을 누르셔서 해당 workspace를 저장해 주시면됩니다.



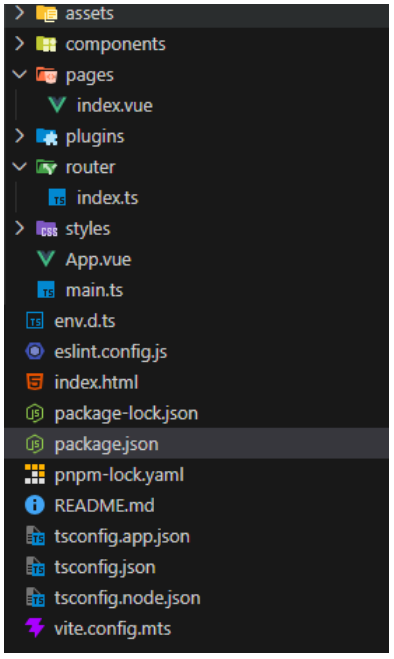


- 정상적으로 저장되었다면 workspace의 이름이 아래 사진과 같이 변경됩니다.



프론트엔드 프로젝트

- frontend1 폴더 안에 package.json이 있을텐데 이 파일이 백엔드 pom.xml과 같은 역할이라고 생각하시면 됩니다.



- 테스트를 위해 명령어를 1개 생성해보겠습니다.

  아래와 같이 hello 명령어를 추가하면

  npm run dev 뿐만 아니라 npm run hello로도 같은 동작이 가능합니다.

- 이제 axios 를 설치합니다. npm install axios —save를 터미널에 입력합니다.

```
PS C:\Users\user\Desktop\CHJ\notion용 사진\test\frontend1> npm install axios --save
```

- 아래와 같이 정상 설치 완료 및 package.json에 추가된 것을 확인할 수 있습니다.

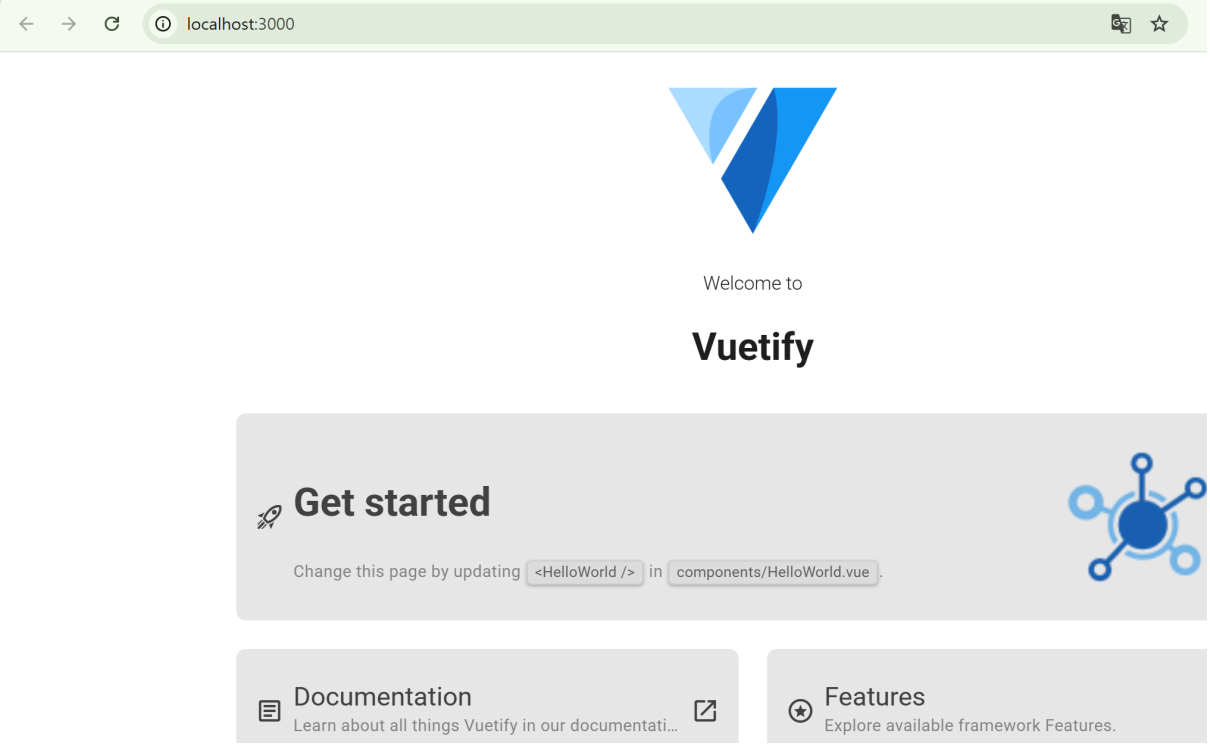
```
PS C:\Users\user\Desktop\CHJ\notion용 사진\test\frontend1> npm install axios --save

added 23 packages, and audited 339 packages in 1s

103 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
PS C:\Users\user\Desktop\CHJ\notion용 사진\test\frontend1>
```

```
"dependencies": {
  "@fontsource/roboto": "^5.2.9",
  "@mdi/font": "7.4.47",
  "axios": "^1.13.6",
  "vue": "^3.5.26",
  "vue-i18n": "^11.2.7",
  "vue-router": "^4.6.4",
  "vuetify": "^4.0.0"
```

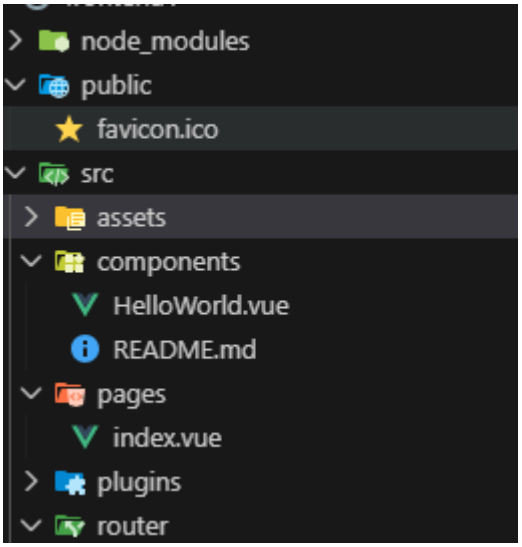
- 터미널에 npm run dev 혹은 npm run hello를 실행하여 정상적으로 프론트 페이지가 실행되는지 테스트를 해봅니다.

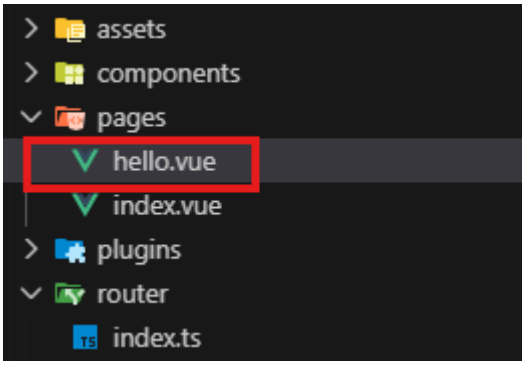
```
PS C:\Users\user\Desktop\CHJ\notion용 사진\test\frontend1> npm run dev

> frontend1@0.0.0 dev
> vite


  VITE v7.3.1  ready in 944 ms

  ➜  Local:   http://localhost:3000/
  ➜  Network: use --host to expose
  ➜  press h + enter to show help
```

- 웹에 http://localhost:3000/ 으로 접속하시면 아래 사진과 같이 페이지가 나타납니다.



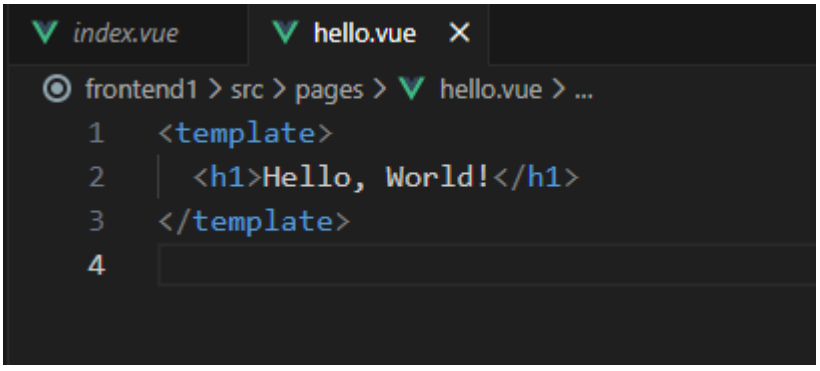
- Vue는 특이하게 서버를 껐다가 킬 필요가 없이 동적으로 로딩이 됩니다.
- pages 폴더 밑에 hello.vue를 생성합니다





- 위와 같이 생성합니다.

- hello.vue 안에 아래 사진과 같이 코드를 작성합니다.



- 작성이 완료되면 http://localhost:3000/hello 로 접속하여 아래 사진과 같이 Hello, World! 가 출력되는지 확인합니다.

> 💡 COC(Convention over Configuration)에 의해서 주소가 /hello로 잡히게 됩니다.

Axios를 활용하여 백엔드와 프론트엔드 연결하기

- 먼저 ./mvnw spring-boot:run 을 통해 백엔드 서버를 열어줍니다.

```
PS C:\Users\user\Desktop\CHJ\notion용 사진\test\example1> ./mvnw spring-boot:run
        Database JDBC URL [Connecting through datasource 'HikariDataSource (HikariPool-1)']
        Database driver: undefined/unknown
        Database version: 16.4
        Autocommit mode: undefined/unknown
        Isolation level: undefined/unknown
        Minimum pool size: undefined/unknown
        Maximum pool size: undefined/unknown
2026-03-04T12:07:09.863+09:00  INFO 20176 --- [example1] [           main] o.h.e.t.j.p.i.JtaPlatformInitiator
       : HHH000489: No JTA platform available (set 'hibernate.transaction.jta.platform' to enable JTA platfor
m integration)
2026-03-04T12:07:09.906+09:00  INFO 20176 --- [example1] [           main] j.LocalContainerEntityManagerFacto
ryBean : Initialized JPA EntityManagerFactory for persistence unit 'default'
2026-03-04T12:07:10.119+09:00  WARN 20176 --- [example1] [           main] JpaBaseConfiguration$JpaWebConfigu
ration : spring.jpa.open-in-view is enabled by default. Therefore, database queries may be performed during v
iew rendering. Explicitly configure spring.jpa.open-in-view to disable this warning
2026-03-04T12:07:10.555+09:00  INFO 20176 --- [example1] [           main] o.s.b.a.e.web.EndpointLinksResolve
r      : Exposing 1 endpoint beneath base path '/actuator'
2026-03-04T12:07:10.602+09:00  INFO 20176 --- [example1] [           main] o.s.b.w.embedded.tomcat.TomcatWebS
erver  : Tomcat started on port 8080 (http) with context path '/'
2026-03-04T12:07:10.613+09:00  INFO 20176 --- [example1] [           main] com.ubisam.example1.Application
       : Started Application in 3.595 seconds (process running for 3.851)
```

- 그 후 npm run dev 혹은 npm run hello를 통해 프론트엔드 서버도 미리 열어줍니다.

```
PS C:\Users\user\Desktop\CHJ\notion용 사진\test\frontend1> npm run hello
  VITE v7.3.1  ready in 689 ms

  ➜  Local:   http://localhost:3000/
  ➜  Network: use --host to expose
  ➜  press h + enter to show help
```

- 이제 hello.vue로 가서 아래 코드를 작성해줍니다.

  아까 설정한 /api/helloes/search 로 연결해줍니다.

  또한 백엔드 서버를 열었을때 Tomcat started on port 8080 이기 때문에

  localhost:8080 으로 이어줍니다.

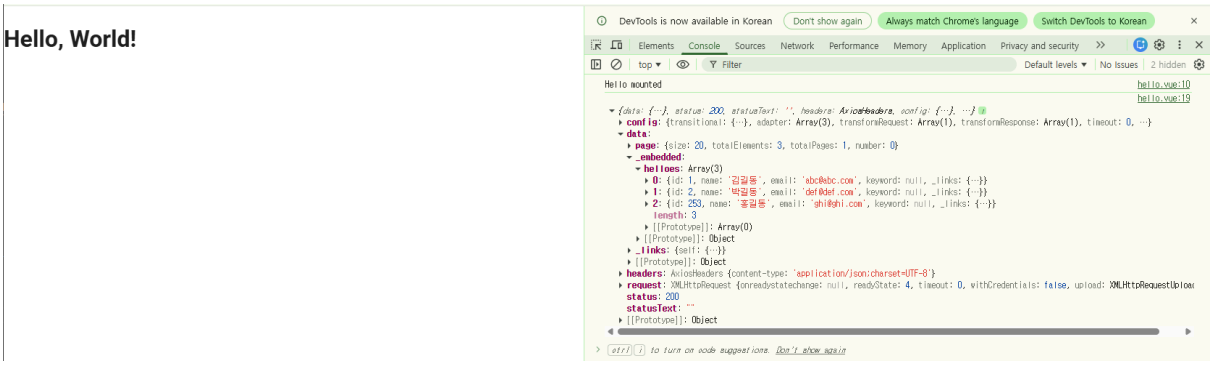
```
<script>
  import axios from 'axios'

  export default {
    mounted () {
      console.log('Hello mounted')
```

```
      axios({
        method: 'post',
        url: 'http://localhost:8080/api/helloes/search',
        data: {
          keyword: '',
        },
      })
        .then(res => {
          console.log(res)
        })
        .catch(error => {
          console.error('목록 가져오기 실패:', error)
        })
    },
  }
</script>
```

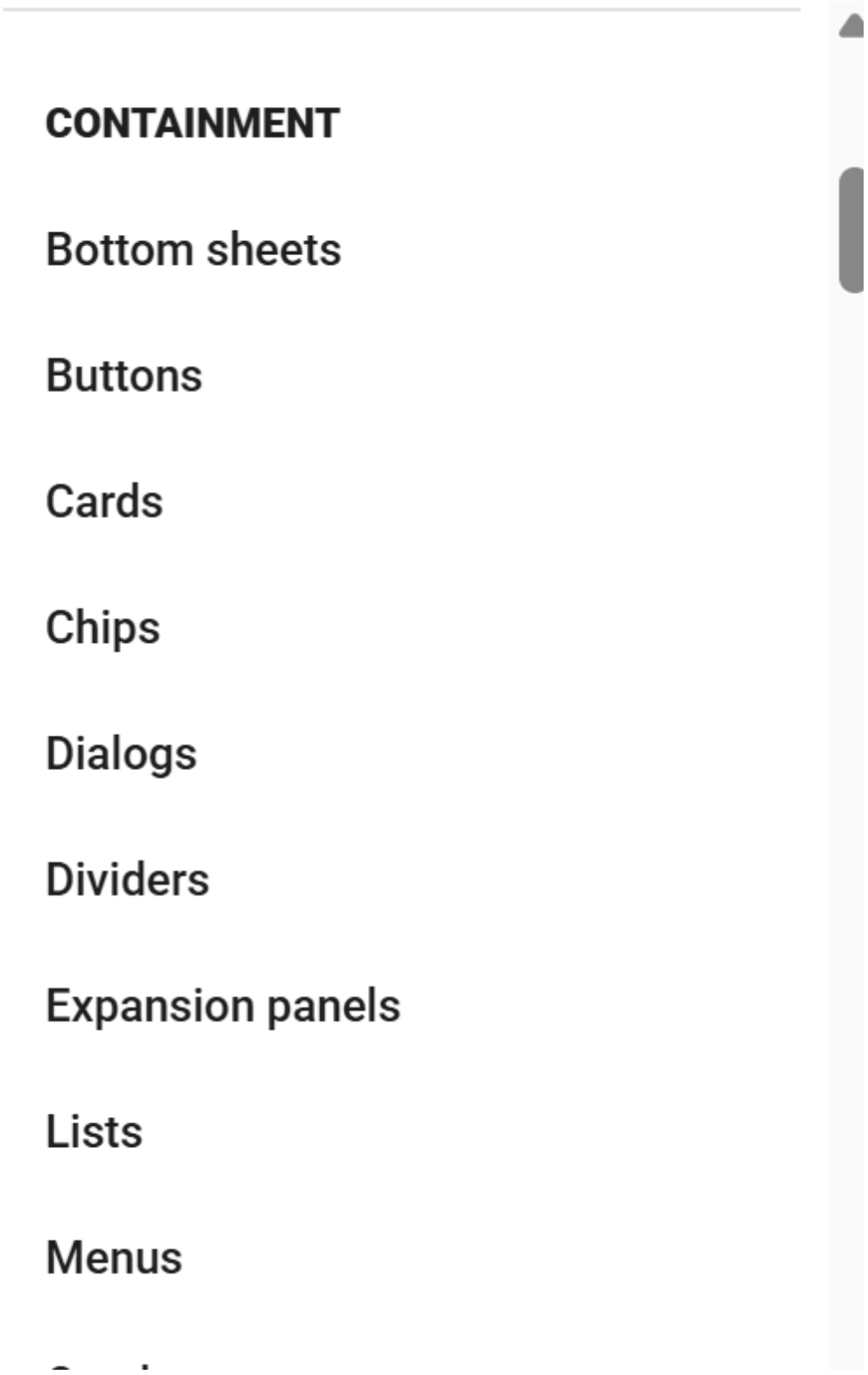
```vue
<template>
  <h1>Hello, World!</h1>
</template>

<script>
  import axios from 'axios'

  export default {
    mounted () {
      console.log('Hello mounted')
      axios({
        method: 'post',
        url: 'http://localhost:8080/api/helloes/search',
        data: {
          keyword: '',
        },
      })
        .then(res => {
          console.log(res)
        })
        .catch(error => {
          console.error('목록 가져오기 실패:', error)
        })
    },
  }
</script>
```

- 그 후 localhost:3000/hello 에서 개발자 도구를 킨 후

  Console 탭에 가면 아래와 같이 결과를 받아오는 것을 확인하실 수 있습니다.



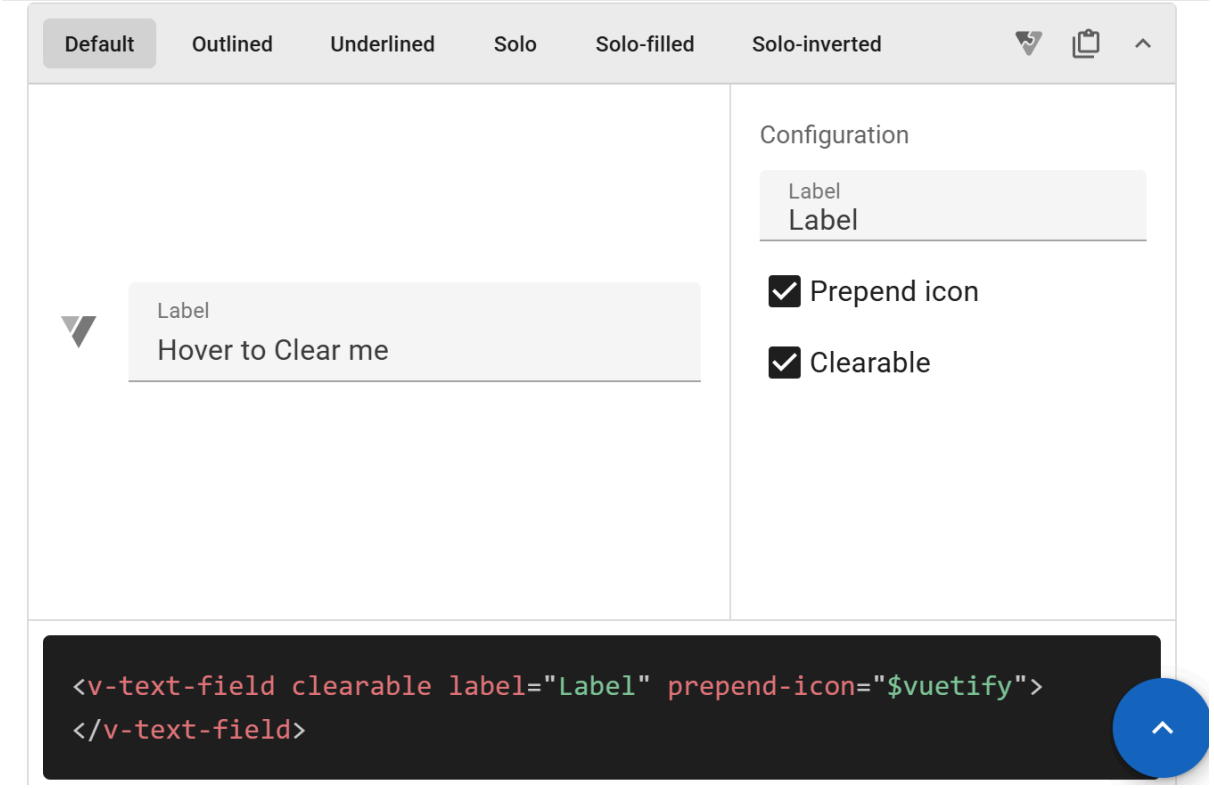
- 꾸미는 방법은 https://vuetifyjs.com/en/components/all/#containment 쪽으로 이동하시면 아래와 같은 메뉴들이 보입니다. 해당 메뉴를 사용하여 프론트를 꾸미면 됩니다.



- 예시로 https://vuetifyjs.com/en/components/text-fields/#usage

  를 사용해보겠습니다.

  아래와 같이 옵션을 선택하고 밑에 코드를 복사합니다.



- 그 후 hello.vue에 template 안에 해당 코드를 입력합니다.

```
<template>
  <h1>Hello, World!</h1>
  <v-text-field clearable label="Label" prepend-icon="$vuetify">

  </v-text-field>
</template>
```

- 그럼 hello 화면에서는 아래 사진과 같이 text-field가 추가된 것을 확인하실 수 있습니다.

ESLint 빨간줄 없애기

- 아래 사진과 같이 빨간줄이 뜨시는 분들을 위해 추가합니다.

```
export default {
mounted () {
    console.log('Hello mounted')
    axios({
    method: 'post',
    url: 'http://localhost:8080/api/helloes/search',
    data: {
        keyword: '',
    },
    })
    .then(res => {
        console.log(res)
    })
    .catch(error => {
        console.error('목록 가져오기 실패:', error)
    })
},
}
</script>
```

- 해당 설정은 eslint.config.js에서 설정을 오른쪽과 같이 바꿔주시면 해결됩니다.

```
import vuetify from 'eslint-config-vuetify'

export default vuetify({
  ts: true,
})
```

```
// import vuetify from 'eslint-config-vuetify'

// export default vuetify({
//   ts: true,
// })
export default []
```

게시판 만들기 (테스트용)

- 아래는 참고한 링크입니다.

---

https://vuetifyjs.com/en/components/all/

https://www.npmjs.com/package/@mdi/font

https://mdisearch.com/

https://vuetifyjs.com/en/features/rules/#guide

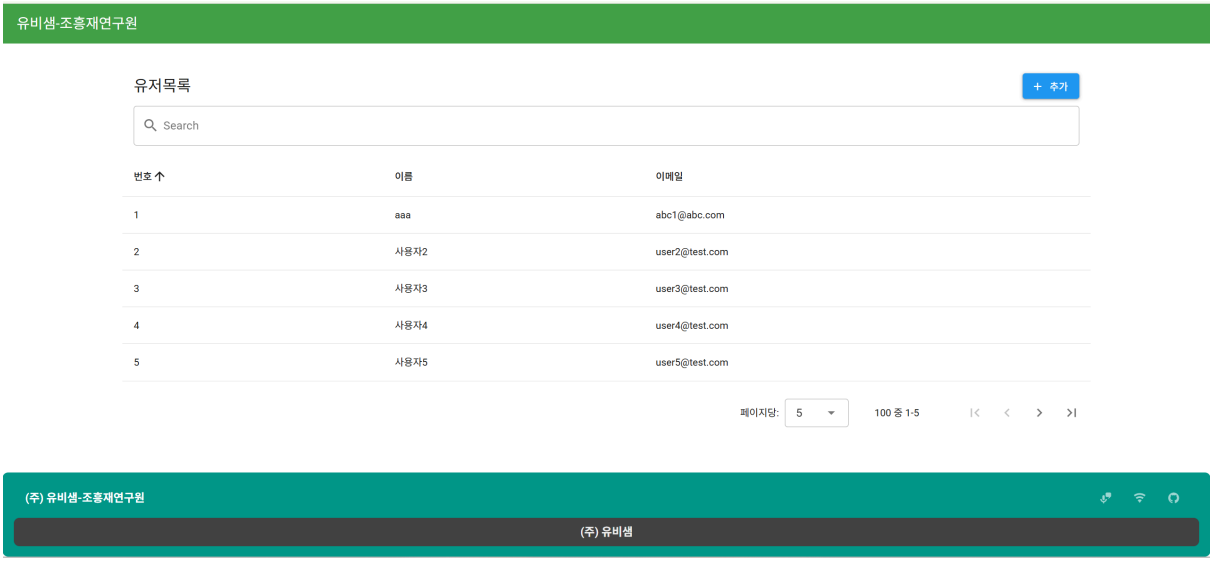
https://wikidocs.net/330816

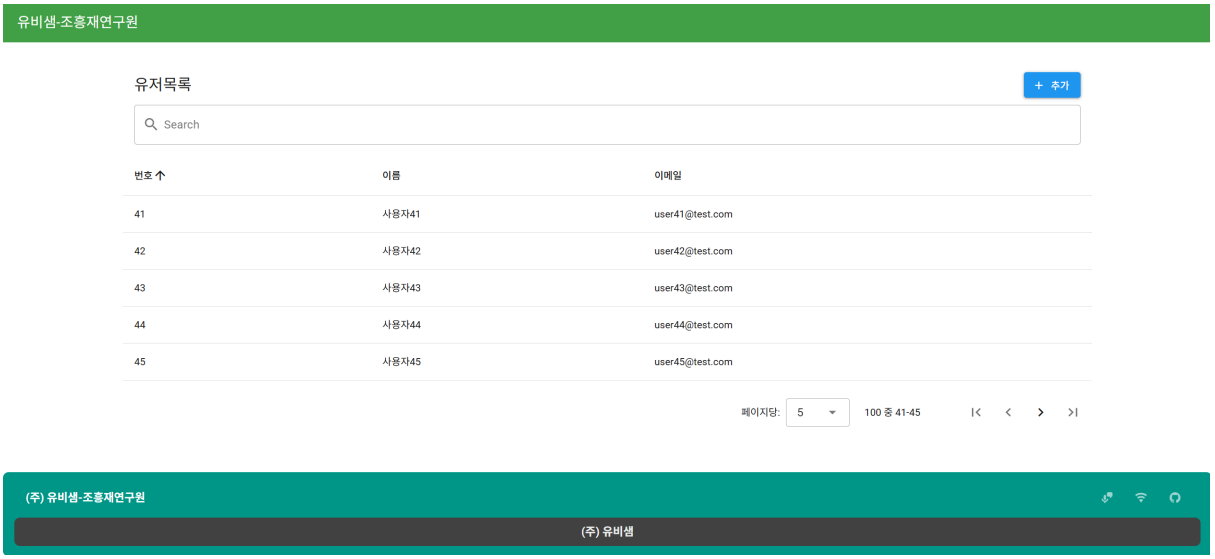
https://www.youtube.com/watch?v=egt_Judt66I

https://ko.vuejs.org/guide/essentials/template-refs

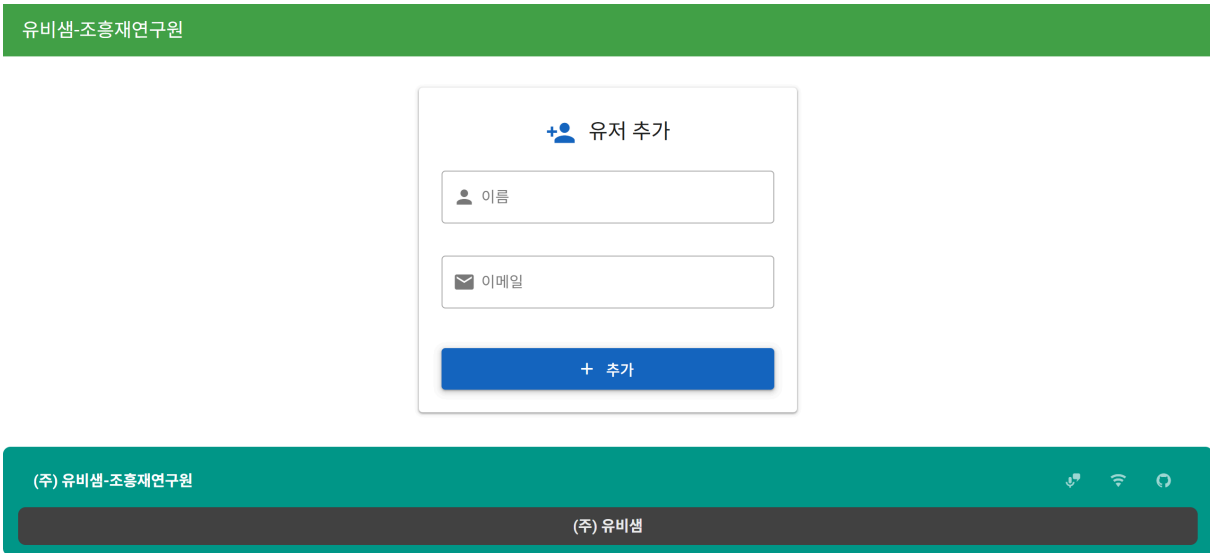
https://kmuhan-study.tistory.com/16

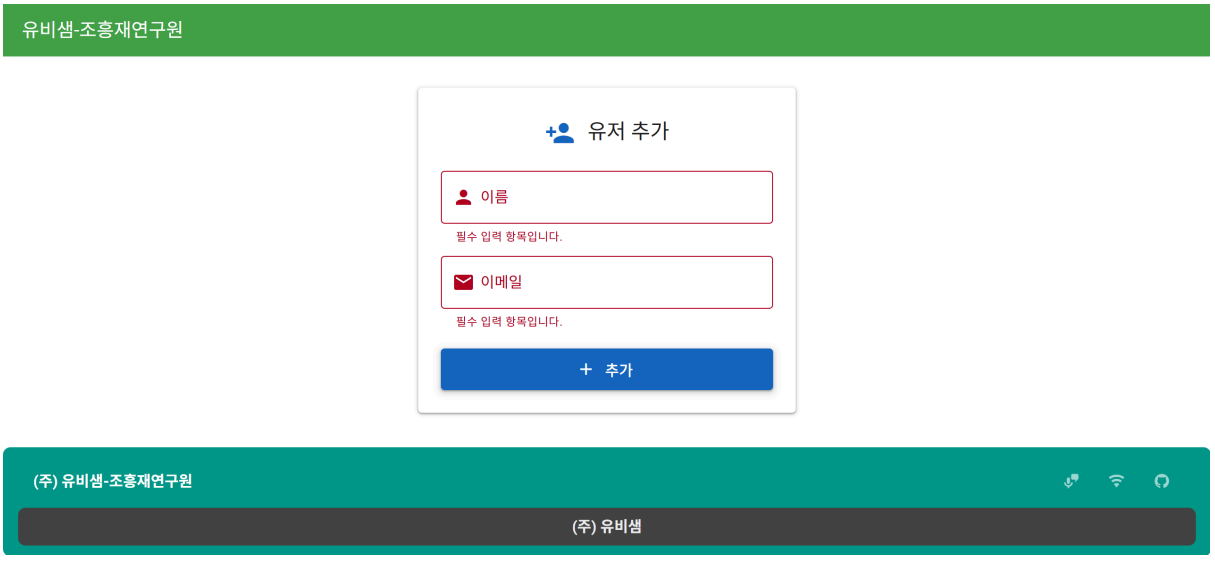
- 목록페이지 입니다



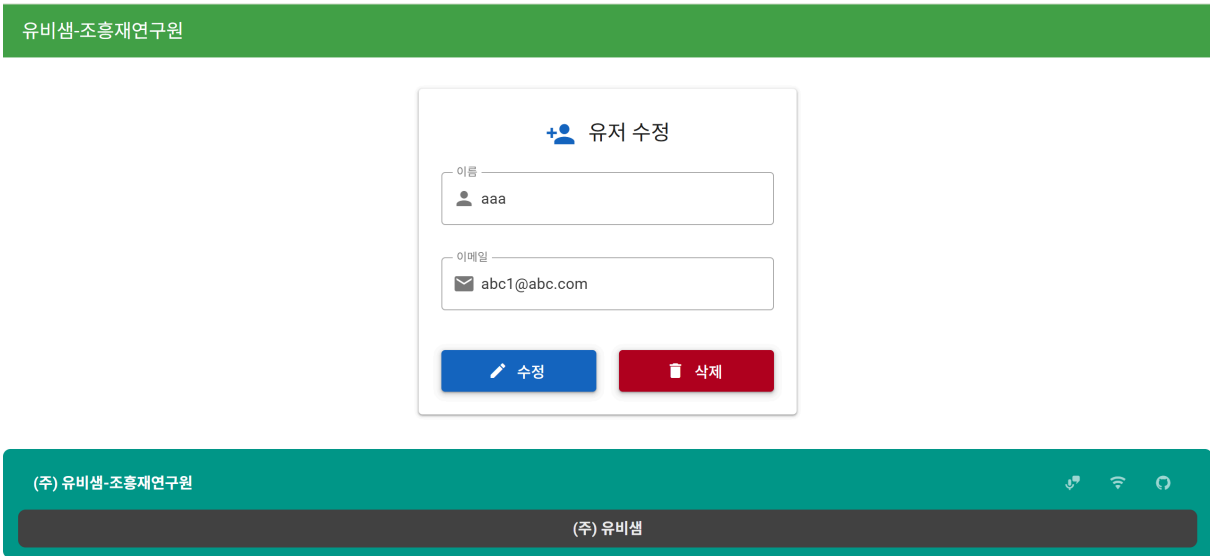

유비샘-조흥재연구원
유저목록
+ 추가
Search
번호
이름
이메일
41
사용자41
user41@test.com
42
사용자42
user42@test.com
43
사용자43
user43@test.com
44
사용자44
user44@test.com
45
사용자45
user45@test.com
페이지당:
5
100 중 41-45
(주) 유비샘-조흥재연구원
(주) 유비샘

- 유저 추가 페이지 입니다

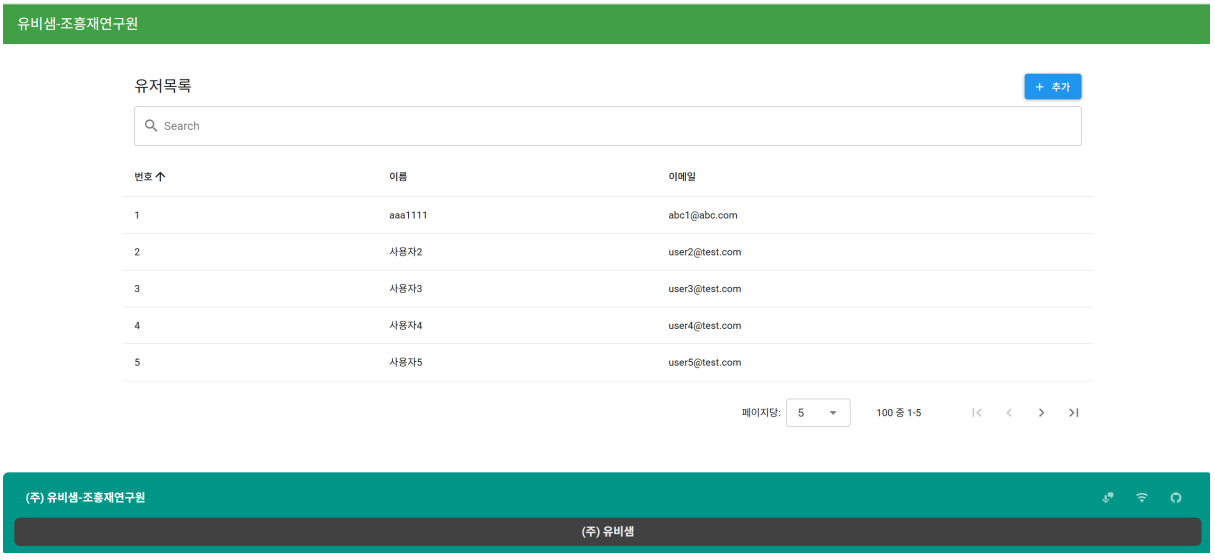

유비샘-조흥재연구원
유저 추가
이름
이메일
+ 추가
(주) 유비샘-조흥재연구원
(주) 유비샘


유비샘-조흥재연구원
유저 추가
이름
필수 입력 항목입니다.
이메일
필수 입력 항목입니다.
+ 추가
(주) 유비샘-조흥재연구원
(주) 유비샘

- 유저 수정 페이지 입니다


유비샘-조흥재연구원
유저 수정
이름
aaa
이메일
abc1@abc.com
수정
삭제
(주) 유비샘-조흥재연구원
(주) 유비샘

유비샘-조흥재연구원
유저 수정
이름
aaa1111
이메일
abc1@abc.com
수정
삭제
(주) 유비샘-조흥재연구원
(주) 유비샘


유비샘-조흥재연구원
유저목록
+ 추가
Search
번호
이름
이메일
1
aaa1111
abc1@abc.com
2
사용자2
user2@test.com
3
사용자3
user3@test.com
4
사용자4
user4@test.com
5
사용자5
user5@test.com
페이지당:
5
100 중 1-5
(주) 유비샘-조흥재연구원
(주) 유비샘

이렇게 3월 3일 세미나에 대한 내용 공유였습니다.

제 소스 파일은 github에 올려놨습니다.

https://github.com/ubisam-heung/myownstudy-backend

https://github.com/ubisam-heung/myownstudy-frontend

긴 글 이였는데도 읽어주셔서 감사합니다 😊